

sintesi

1) per ogni registro (separabili) servirà cosa succede negli stati

2) var. comb.: sostituisci gli stati con i bit (es. se ho solo 1 condizione ho bisogno solo di 1 bit, se sono 3 ho bisogno di 2 bit ecc.) → ogni assegnamento a reg. è un valore di bit diverso

↓
1 var. 6
↓
ho bisogno di
2 var. 6

→ non considero la parte di STAR

→ ricordati di mettere anche lo stato di hold ($REG \leq REG$)

3) var. condizionamento

guarda la parte di STAR

sostituisci le condizioni con delle variabili $c_i = \text{condizione}$
trasformare in espressioni logiche

assegna bi: assign { } =

(STAR == S0) ? valore delle var. b in quello stato
(STAR == S1) ?

4) separo modulo ABC: wire, in, out della rete finale

parte operativa

- assign c
- o reg op → reg op
- o rc che ne pilota gli ingressi/gen var. condiz.
- o input: clock, reset, ..., var comando
- o output: var condiz

parte di controllo

- o reg STAR[2:0]
- o input: var. condiz. (c), clock, reset
- o output: var. comando (b)
→ assign b

guarda nel blocco STAR della PC
per quale ci cambia stato
portando allo stato M-addr
↑

5) tabella ROM: M-addr = stato in binario, $b_i \dots b_0$, c-eff, M-addr-T, M-addr-F
se non ci sono condizioni c-eff = X

2023-07-08

interf RC : moltiplica per 5 leggendo da A e inviando su B $\rightarrow$ formula $2.5 = (2.1) + (2.4)$

input 2 a 8 bit
output m a 16 bit (risultato)

estendo 2 $\rightarrow$ 21-ext = 002 = 2.1

estendo 2 $\rightarrow$ 24-ext = 200 = 2.4

summa 21-ext + 24-ext $\rightarrow$ è a 10 bit perché 24-ext sta su 10 bit

assign m = { 5'hex, c_out, sum } $\rightarrow$ 5 bit di padding + c_out + 10 bit di somma (carry)
su 16 bit

RS

2 interfacce A, B ^{con handshake} _{senza handshake}

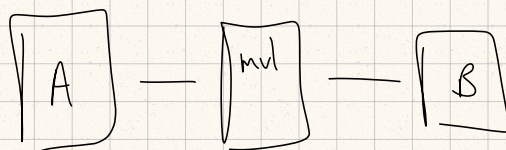
- legge byte da A e moltiplica per 5

- output prodotto tramite B

A: offset = 0x100
B: offset = 0x120

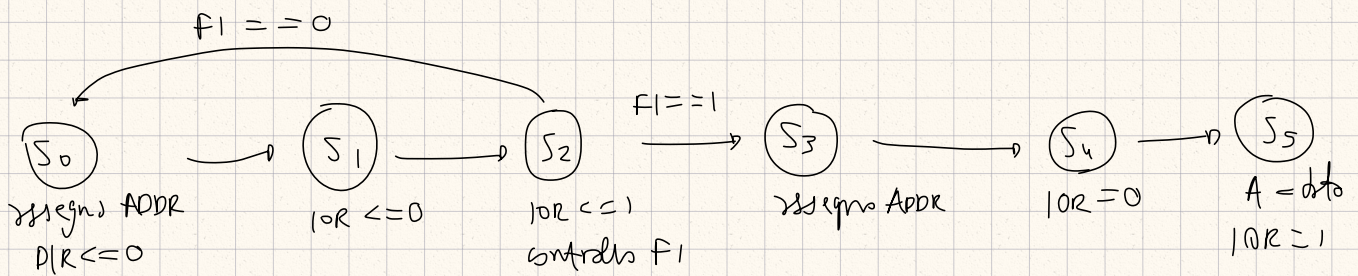
A: RSR montato all'ind. int. 0

RKR // // 1

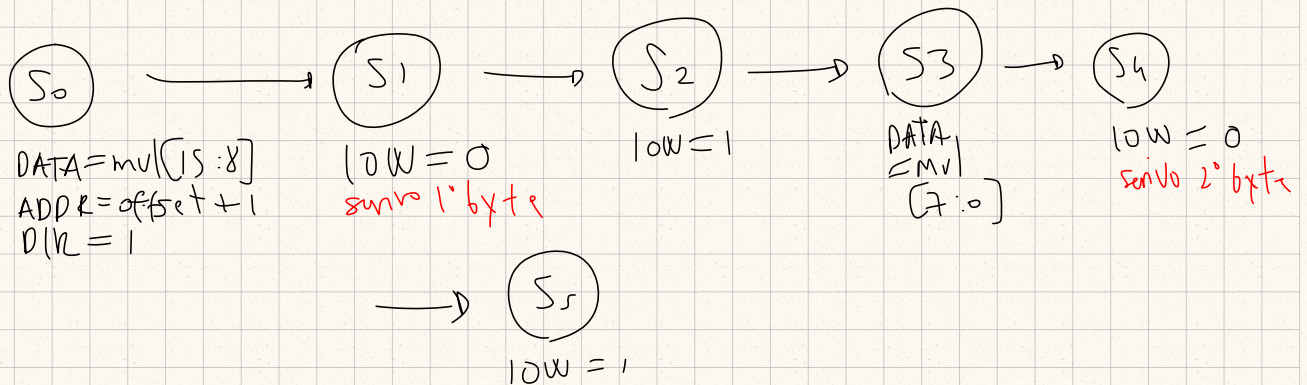


descrizione RS

stati in A:



poi vado negli stati SB

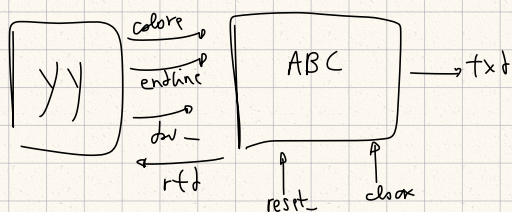


interfaccie

lettura: prepara dati $\rightarrow IOR_{\leq 0} \rightarrow IOR_{\leq 1}$
($ADDR \leq offset$)

scrittura: prepara dati $\rightarrow LOW_{\leq 0} \rightarrow LOW_{\leq 1}$
 $DIR \leq 1$

2021-01-26 vedi pg 51 dispensa calcolatore



ABC costruisce $\{N, B\}$ o p'hoo se endline = 1 e inizia una nuova sequenza
↓
num consec.
pixel di
colore B

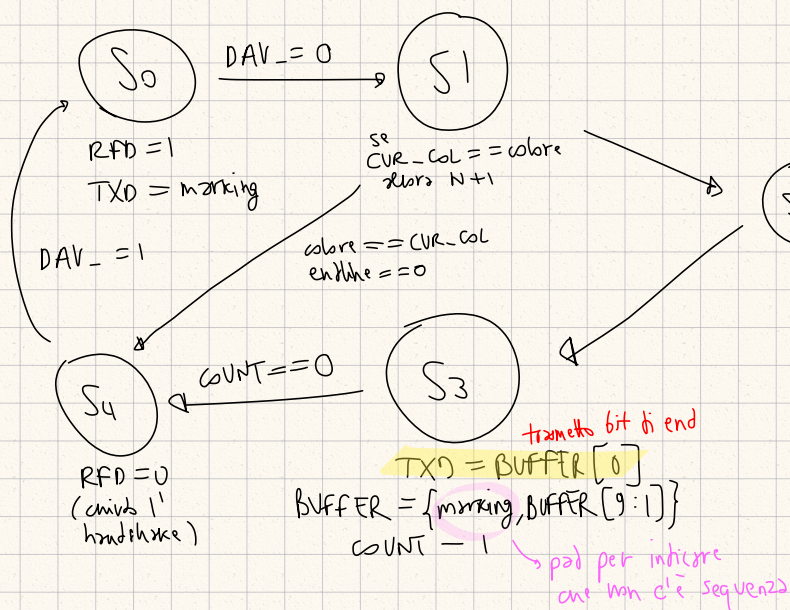
registri

RFD
 TXD
 N
 COUNT → contatore di quante caratteri sei
 CUR_COL → colore corrente
 BUFFER → dati da trasmettere

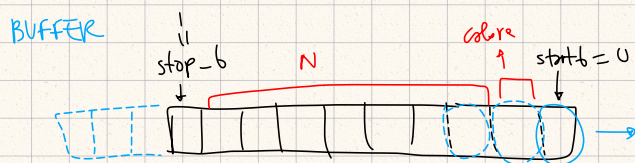
localparam

ner = 0
 bianco = 1
 marking = 1 → padding
 start_b = 0 → indica l'inizio di un messaggio
 stop_b = 1 → fine msg = marking

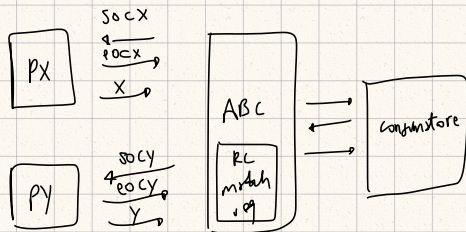
stati $\left. \begin{array}{l} \text{COUNT} = 0 \\ \text{COLORE} = 1 \text{ (nero)} \end{array} \right\} \text{ al reset}$



BUFFER =
 se endline = 1 → {stop_b, N, CUR_COL, start_b}
 else → {stop_b, 8'h00, start_b}
 COUNT = 9 → reset il counter dei byte consecutivi
 CUR_COL = colore
 N = (endline = 0)? 1 : 0



2025-09-09

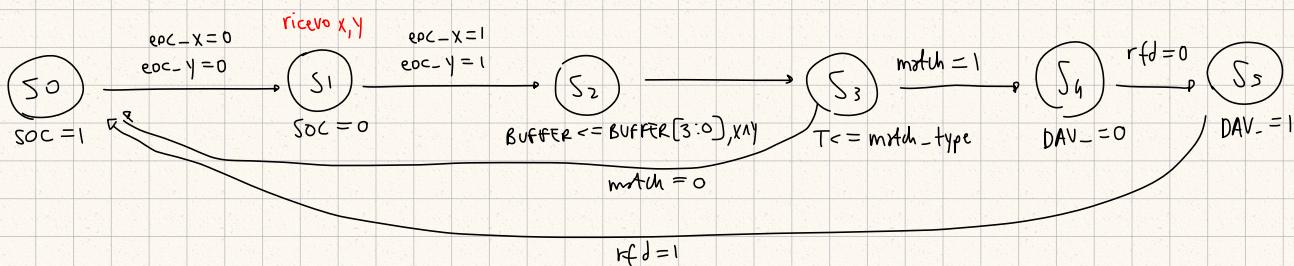


ho bisogno di
BUFFER (sequenziale)
soc
Eoc
match_type → viene dalla RC

0110 controlla gli ultimi 4 bit
1100
0101

ABC:

reset: 60000 0000



rc MATCH-SEQ

input BUFFER output match_type, match

wire type_1

...

sign type_1 =

match = type_1 | ... |
match_type = (type_1 ? 1, type_2 ? 2 ...)

2024-01-10

A/D $\rightarrow X, Y$

ABC: $(X + Y)^2$ invia al consumatore (serve handshake rfd / dav_)

sintesi RC quadrato della somma

sum x, y

x e y a 8 bit

$x + y$ al massimo sta su 9 bit

$$2^8 \cdot 2^8 = 2^{16}$$

$(x + y)(x + y)$ sta su massimo 18 bit

$$(x + y)^2 = x^2 + 2xy + y^2 \rightarrow \text{somma di prodotti}$$

$$\begin{array}{ccc} x \cdot x & + & 2x \cdot y & + & y \cdot y \\ \downarrow & & \downarrow & & \downarrow \\ 16 \text{ bit} & & 16 \text{ bit} & & 16 \text{ bit} \end{array}$$

$x^2 + y^2$ sta su $16 + 1 = 17$ bit

$+ 2xy$ su 18 bit

1) mul-add-not : $x \cdot x$

2) mul-add-not : $y \cdot y$

3) mul-add-not : $x \cdot y$

4) add : $x \cdot x + y \cdot y$

5) add : $x \cdot x + y \cdot y + \boxed{x \cdot y \cdot 2}$

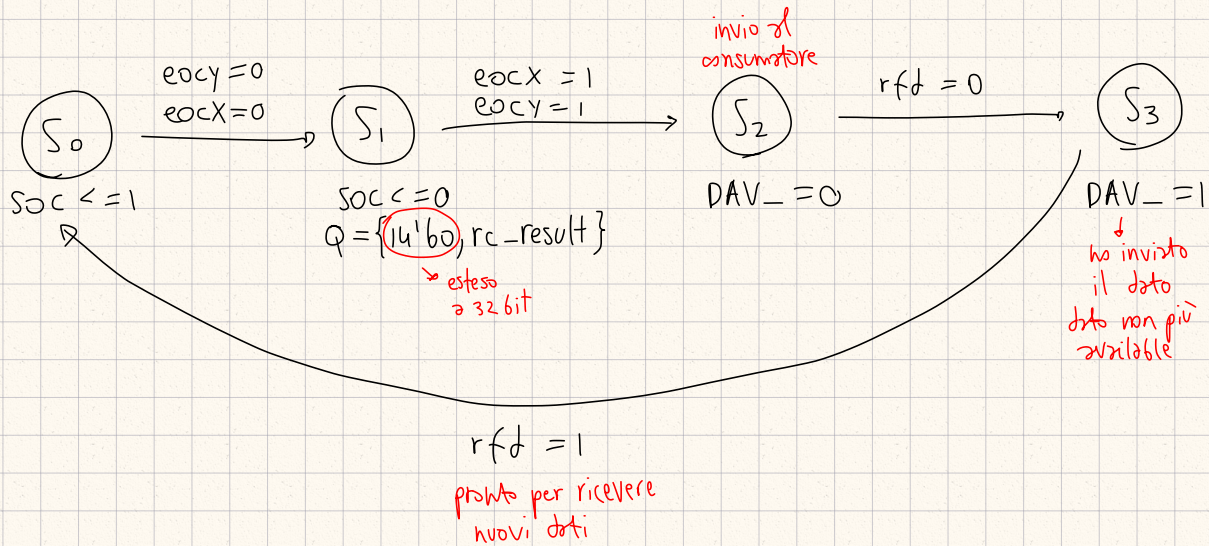
Shift a sinistra
per moltiplicare xy
per 2

descrizione RS

stati

al reset

DAV₋ = 1
SOC = 0
STAR = S₀



Sintesi RS

Variabili di comando

DAV₋ {b₁, b₀}

2'b00	= 0
2'b01	= 1
2'b10	hold

SOC {b₃, b₂}

2'b00	= 0
2'b01	= 1
2'b10	hold

Q {b₄}

1'b0	= 0
1'b1	= hold

Variabili di condizionamento

C₀: eocx == 0 & eocy == 0
C₁: eocx == 1 & eocy == 1
C₂: rfd == 1

2023-02-13 (interfaccia)

$OK=1$ se test passato (polling tra ABC $\rightarrow$ A)

quando $OK=1$ ABC trasferisce a B

B: interfaccia all'offset $h'0ABC$

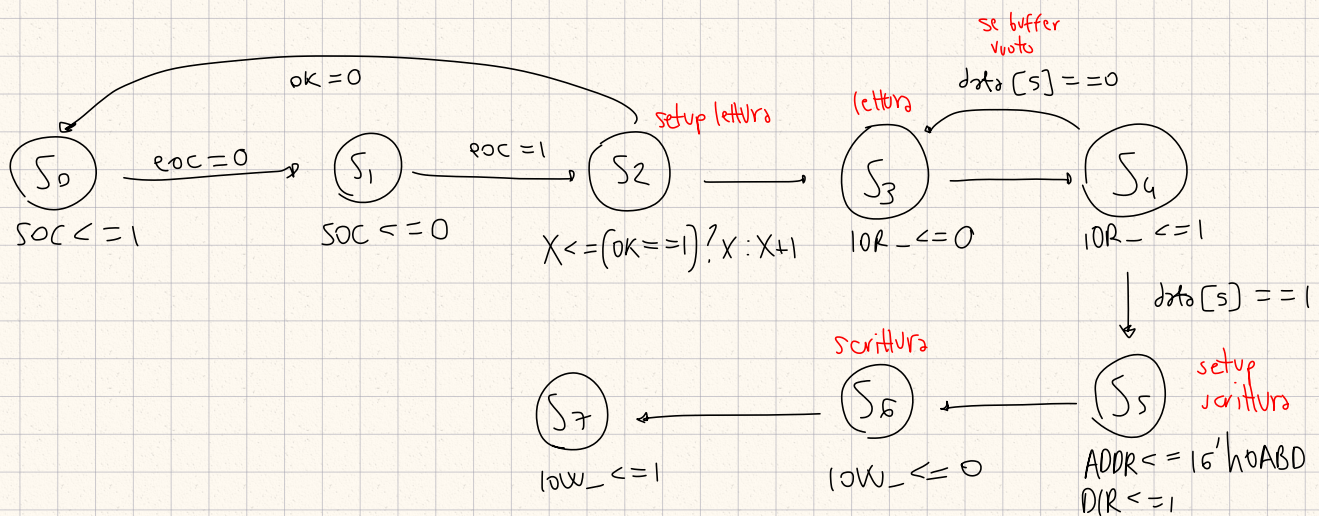
descrizione RS

reset:

$SOC \leq 0$
 $STAR \leq S_0$

$IOR_{-} \leq 1$
 $IOW_{-} \leq 1$

$ADDR \leq h'0ABC$
 $DIR \leq 0$



(bus)

data = se $DIR=1$ vale x (scrittura)

se $DIR=0$ è in alta impedenza ("stacco" l'uscita)

2024-02-13

X, Y in CR su 8 bit X, Y

RC $\rightarrow$ 2 consumatori

Sintesi RC

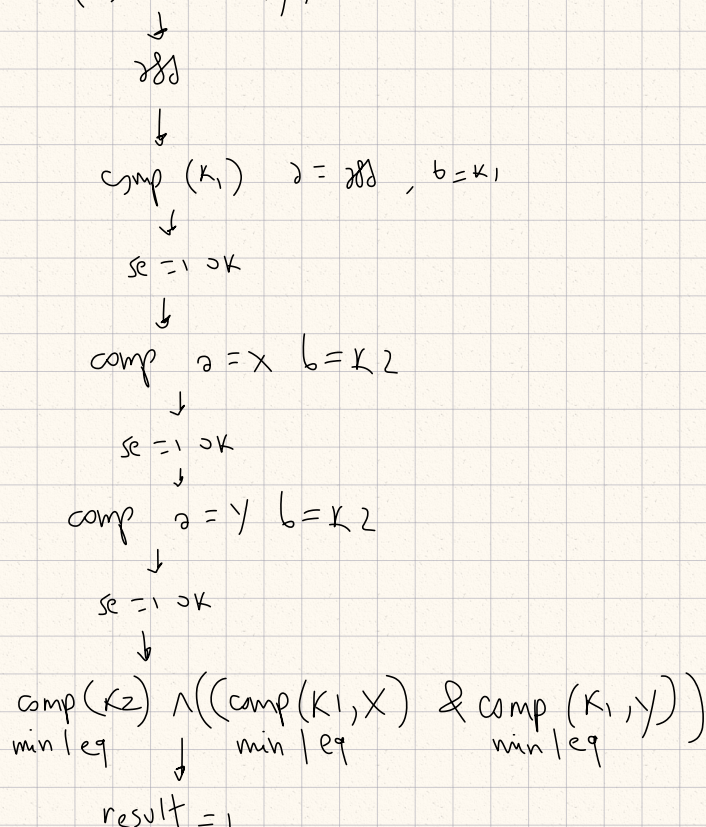
uscita z 1 se p interno, 0 altrimenti (zona bianca)
o linee tratteggiate

$K_1 = 64$

$K_2 = 48$

$|x| + |y| \leq K_1$ e $|x| \leq K_2$ AND $|y| \leq K_2$

$obs(x)$ $obs(y)$



Desc RS

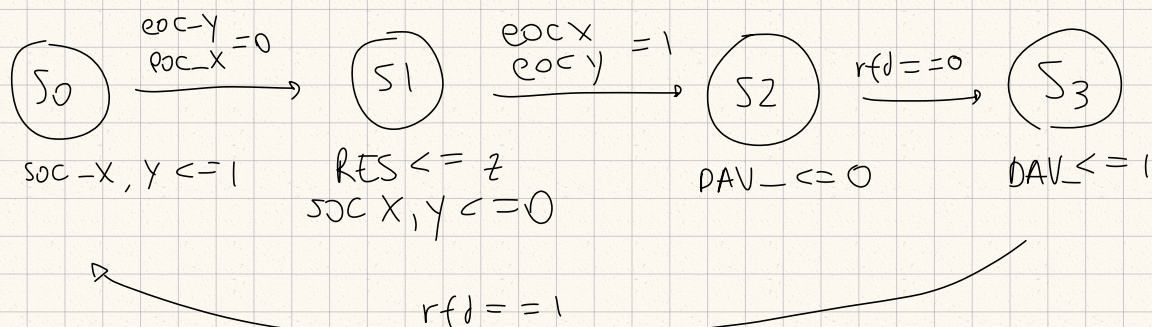
Reset

$SOC-X = 0$

$SOC-Y = 0$

$DAV- = 1$

$STAR <= 50$



2024-09-10

esercizio sul ricevitore seriale

$rx_d(1 \text{ byte}) \rightarrow X_n$ in compl. alla radice

$$S_n = X_n + X_{n-1} \\ (\text{con segnale di ow})$$

$$X_0 = 0$$

se S_n sta su 1 byte setto out a 1 per 1 ciclo
se non sta // out a 1 //

trasmissione:

$rx_d = \text{mark}$ (assenza di trasmissione) per $20 \cdot 0 + T$ di clock

$rx_d = \text{space}$ da $2 \cdot 7$ T di clock $\rightarrow$ trasmetto 1

$rx_d = \text{space}$ da $11 \cdot 15$ T di clock $\rightarrow$ trasmetto 0

byte trasmessi a partire dal LSB

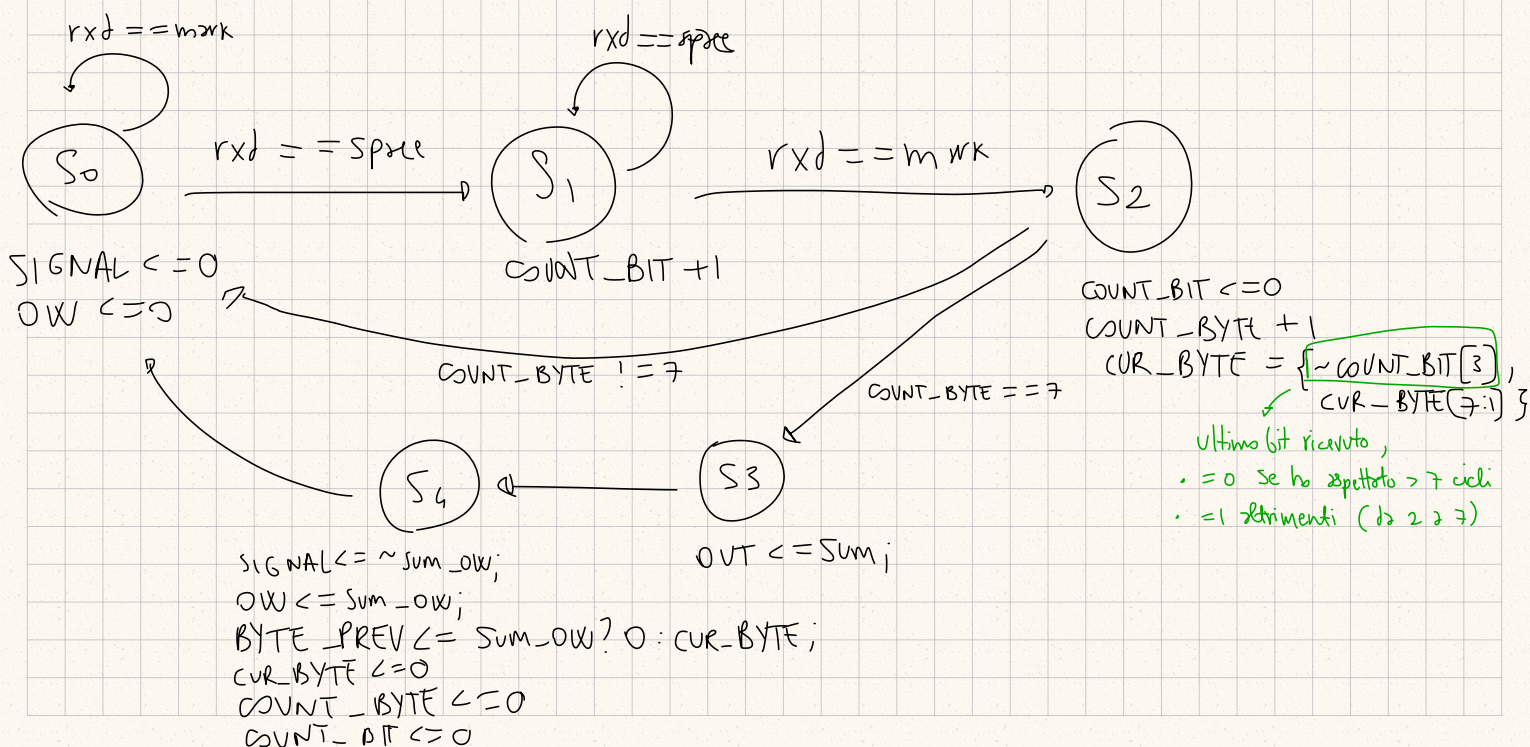
sintesi R4: basta usare un adder con $N=8$

descrizione R5

aspetto 2 cicli

// 7 cicli $\rightarrow$ dopo 7, trasmetto 0?

al reset:
 $CUR_BYTE, BIT = 0$
 $COUNT_BYTE, BIT = 0$
 $STAR \leq S_0$



2024-06-24

al reset:
 $x=0$
 $y=0$

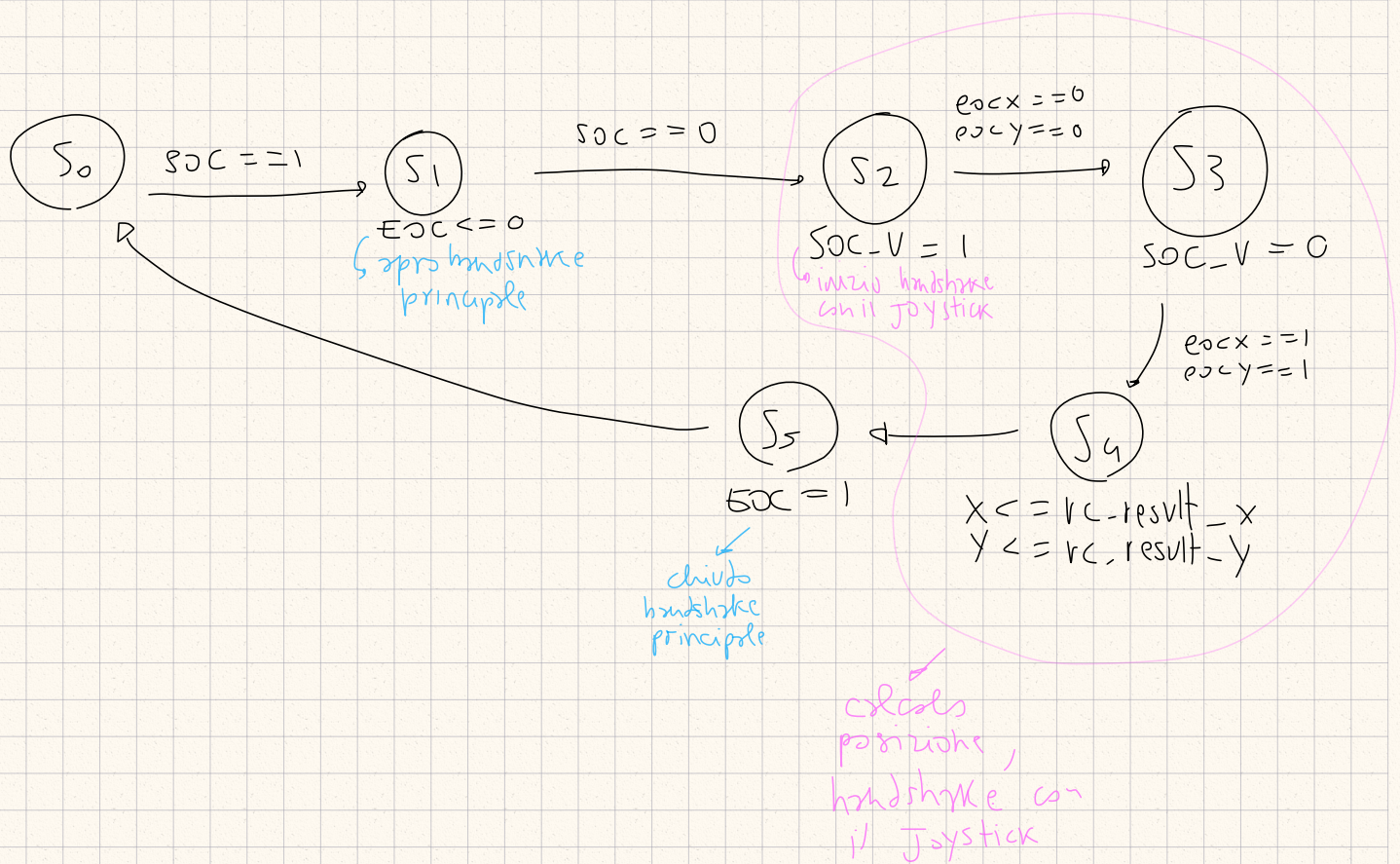
sintesi RC

semplice somma di v_x/v_y alla coordinata, se $ow=1$ non cambio il valore

descrizione RS

al reset:

$x=0$ $y=0$; $Eoc \leq 1$; $SOC_V=0$; $STAR \leq 50$;



2024-07-16

A/D binario unipolare

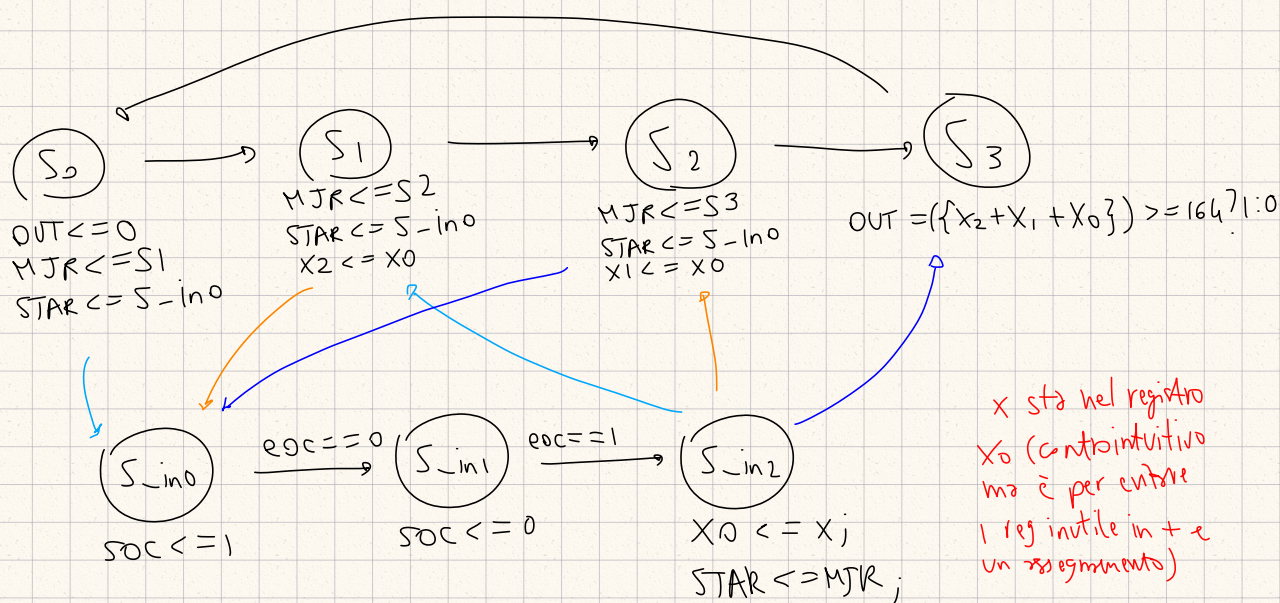
1. preleva 3 campioni dal produttore
2. $S = x_2 + x_1 + x_0$ x su 8 bit
3. $out = 1$ se $S \geq 164$

descr RS

registri SOC, OUT, x_2, x_1, x_0

devo usare sottoprogr. quindi serve MJR

al reset: $SOC = 0$ $OUT = 0$ $STAR = S_0$



sintesi RS

gestione MJR:

Output della PO + registro nell. PO

input della PC

Wire nel modulo principale

nella ROM va aggiunto il p-type (= 1 se salto guidato da MJR)